Reliability-Aware and Energy-Efficient Workflow Scheduling in IaaS Clouds

Lingjuan Ye, Yuanqing Xia, *Senior Member, IEEE*, Siyuan Tao, Ce Yan, Runze Gao, and Yufeng Zhan

***Abstract*—Nowadays, more and more workflow applications with different computing requirements are migrated to clouds and executed with cloud resources. Workflow scheduling becomes a critical problem in the cloud environment, which focuses on meeting various quality of service (QoS) constraints. Workflow reliability and energy consumption are two essential parts in clouds and minimizing energy consumption for scheduling workflow with the reliability constraint is a challenging issue. In response to the challenge, we propose a workflow scheduling algorithm named REWS to reduce energy consumption and satisfy workflow reliability constraints. In REWS, a new sub-reliability constraint prediction strategy is adopted to break down the workflow reliability constraint to task sub-reliability constraints and the effectiveness of this strategy is proved. Moreover, an update method is adopted to adjust the task sub-reliability constraint for reducing energy consumption. In addition, a brief system framework which consists of five parts: workflow analyzer, reliability decomposer, resource manager, workflow scheduler and feedback processer is built to support the algorithm implementation of REWS. We conduct the experiments using both synthetic data and real-world data to evaluate the proposed REWS approach. The results demonstrate the superiority of REWS as compared with the state-of-the-art algorithms.**

***Note to Practitioners*—Workflow scheduling is a challenging issue in emerging trends of the cloud environment that focuses on satisfying various QoS constraints. In this paper, we investigate a reliability-aware and energy-efficient workflow scheduling problem in cloud computing. A novel workflow scheduling algorithm called REWS, is designed to reduce the energy consumption and meet the workfolw reliability constraint. The basic idea of REWS is to divide the workflow reliability constraint into task sub-reliability constraints and schedule tasks with an energy-efficient scheduling strategy. We conduct the experiments to evaluate the proposed REWS and the results demonstrate that REWS outperforms the state-of-the-art algorithms.**

***Index Terms*—Cloud computing, workflow scheduling, energy consumption, workflow reliability constraint.**

Manuscript received 18 November 2021; revised 22 February 2022 and 16 May 2022; accepted 20 June 2022. Date of publication 19 August 2022; date of current version 3 July 2023. This article was recommended for publication by Associate Editor M. Robba and Editor B. Vogel-Heuser upon evaluation of the reviewers' comments. This work was supported in part by the National Key Research and Development Program of China under Grant 2018YFB1003700, in part by the National Natural Science Foundation of China under Grant 61836001 and Grant 62102022, and in part by the Graduate Interdisciplinary Innovation Project of Yangtze Delta Region Academy of Beijing Institute of Technology (Jiaxing) under Grant GIIP2021-001. *(Corresponding author: Yuanqing Xia.)*

Lingjuan Ye and Siyuan Tao are with the School of Mathematics and Statistics, Beijing Institute of Technology, Beijing 100081, China (e-mail: lingjuanye@126.com).

Yuanqing Xia, Ce Yan, and Runze Gao are with the School of Automation, Beijing Institute of Technology, Beijing 100081, China (e-mail: xia_yuanqing@bit.edu.cn).

Yufeng Zhan is with the School of Automation, Beijing Institute of Technology, Beijing 100081, China, and also with the Yangtze Delta Region Academy, Beijing Institute of Technology (Jiaxing), Jiaxing 314019, China.

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TASE.2022.3195958.

Digital Object Identifier 10.1109/TASE.2022.3195958

I. Introduction

CLOUD computing is a new way to provide a powerful computing infrastructure with virtualized technologies which can dynamically allocate cloud resources for executing user's tasks [1], [2]. These cloud resources, such as CPU, RAM, disk storage and bandwidth, are elastically used to process computing applications according to requests of users with different types of virtual machines (VMs) [3], [4]. This way brings great convenience to users and improves work efficiency [5]. Recently, the workflow applications, such as earthquake science workflow Cybershake, bioinformatics workflow Epigenomics, Gaussian elimination workflow, and fast Fourier transform workflow in mathematics, have been successfully executed in the cloud platform.

However, due to the increasing scale and diversity of workflow requirements, more and more cloud resources are required to execute workflow [6] and the energy consumption of clouds becomes huger and huger [7], [8]. In July 13, 2015, the energy consumption of the heterogeneous computing system Tianhe-2 is almost 17,808 KW, which is about 156 million yuan (USD 25 million) a year [9]. Furthermore, according to nature resources defense council (NRDC) [10], the consumption of U.S. cloud date centers is about 91 billion KWh of electricity in 2013 and increases to 140 billion in 2020. Therefore, the energy efficiency of clouds is considered the chief concern for service providers in recent years [11], [12]. It becomes a important challenge to obtain the optimal energy consumption in clouds.

On the other hand, cloud computing is composed of a family of heterogeneous VMs connected by communication network or commercial network. These VMs have different infrastructure and management technologies and are organized by different hosts. In this case, various failures of cloud resources are easy to occur, where hardware failure, resource missing failure, timeout failure, and network failure are the most common failures [13]. As a result, it is inevitable that the reliability of cloud computing decline. In addition, a workflow application usually consists of a large number of precedence constrained tasks [14]–[16]. These tasks are connected by controlflow dependencies and data-flow and executed with

1545-5955 © 2022 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

cloud resources to finish workflow [17]. Any of these tasks failed to execute will eventually lead to the collapse of the entire workflow [18]. Therefore, it is the another important challenge to satisfy the workflow reliability constraint in cloud computing [19].

To ensure the workflow reliability, various techniques have been studied [20]–[22], where checkpoint, fault prevention, fault-tolerance and decomposition scheduling are typical methods in clouds. Among these technologies, the most attractive way for meeting workflow reliability constraints is fault-tolerance. In fault-tolerant technologies, backups are used to reduce the failure of task execution with any VM in clouds and replication is the most important technique for tasks backups [22], [23]. With the replication technique, a primary VM is used to execute the primary task and one or many redundant VMs are assigned to task backups. The primary task and backup tasks cooperate to reduce the failure of task execution. In general, replication-based k-fault tolerance techniques ignore the reliability of the primary VM and require k backup VMs to tolerate the task execution failure [22], [24]. However, these backup tasks with the replication technology require a lot of cloud resources to execute them, which results a large number of energy consumption. In this way, larger reliability could cause more energy consumption for workflow executing. Therefore, it is clamant to present an effective workflow scheduling strategy to minimize energy consumption and satisfy the workflow reliability without using replication in cloud computing.

In this paper, we investigate a workflow scheduling problem for minimizing energy consumption and satisfying the reliability constraint in clouds and design a novel workflow scheduling algorithm named REWS to solve this problem. The proposed REWS includes two phases: workflow pre-processing and workflow scheduling. To meet the workflow reliability constraint, a new sub-reliability constraint prediction strategy by making full use of the workflow reliability constraint is design to break down the workflow reliability constraint to each task in the workflow pre-processing phase. Furthermore, to minimize energy consumption, we develop a sub-reliability constraint update method to adjust the task sub-reliability constraint and schedule tasks onto their most suitable VMs for execution in the workflow scheduling stage. More specifically, the main contributions are as follows:

(1) We model the reliability-aware and energy-efficient workflow scheduling in clouds as a constrained optimization problem and propose a reliability-aware and energy-efficient workflow scheduling algorithm to minimize energy consumption and satisfy workflow reliability constraint.
(2) A new sub-reliability constraint prediction strategy is proposed to break down the workflow reliability constraint and obtain the task sub-reliability constraint, and the effectiveness of this strategy is proved. Moreover, an update method is adopted to adjust the task sub-reliability constraint for reducing energy consumption while satisfies the workflow reliability constraint.
(3) We conduct the experiments using both synthetic data and real-world data to evaluate our proposed approach. The results demonstrate the superiority of our approach as compared with the state-of-the-art solutions.

The outline of the paper is organized as follows. We discuss related work in Section II. Section III describes related models and problem formulation. The REWS is explained and the time complexity of REWS is analysed in Section IV. Section V evaluates the performance of REWS. The conclusion is shown in Section VI.

II. Related Work

Workflow scheduling is a challenging issue in emerging trends of the cloud environment [25]–[27]. Many workflow scheduling problems with numerous different constraints and objectives are widely studied for cloud computing in literatures [28]–[30]. In this section, we review related researches with energy consumption scheduling and workflow reliability scheduling.

A. Energy Consumption Scheduling

For the energy consumption workflow scheduling problem, the dynamic voltage frequency scaling (DVFS) technique is one of the most important techniques for reducing energy consumption in clouds [31]. Tang *et al.* [32] proposed a scheduling algorithm with the DVFS technique to schedule workflow task for meeting deadline constraint and reducing the energy consumption in the cloud environment. Safari *et al.* [33] divided the workflow deadline constraint into task sub-deadline constraints based on the dependencies between tasks in workflow and used the DVFS to design a scheduling algorithm for minimizing energy consumption. In [6], a scheduling algorithm with PU-specific DVFS and DRS was studied to minimize the total energy consumption of executing tasks with task deadline constraints. In [34], a scheduling algorithm was proposed to minimize job completion time with energy consumption constraints. In [35], by modeling data sets and tasks as a binary tree, a tree-to-tree task scheduling algorithm was proposed to improve the energy efficiency of a cloud system. In [36], Wang *et al.* proposed a heuristic workflow scheduling algorithm for reducing energy consumption with DVFS technique in clouds. Moreover, the constrained optimization problems of energy consumption and makespan for workflow in clouds were studied by Li [37]. Wen *et al.* considered an energy and cost workflow scheduling algorithm for instance-intensive IoT workflow in clouds [38]. Singh *et al.* designed an energy efficient algorithm with a novel measure to determine the amount of energy which can be conserved by considering a DVS-enabled environment for workflow scheduling in IaaS cloud [39]. These approaches can reduce energy consumption of cloud computing systems with different constraints [40], but the problem of the energy consumption and workflow reliability scheduling problem has not been considered.

B. Workflow Reliability Scheduling

As another category of studies, workflow reliability scheduling problems were very attractive for researchers in recent years. In [41], a reliability-driven grid workflow

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

scheduling algorithm was proposed by Kianpisheh. Zhao *et al.* [42] presented two fault tolerance scheduling algorithms for reliable workflow with minimizing costs in clouds. For real-time scientific workflow, Zhu *et al.* studied a fault-tolerant scheduling algorithm for the problem of elastic resource provisioning in clouds [43]. In addition, Thirumalaiselvan and Venkatachalam considered the workflow scheduling problem in the complex multi-cloud environment and proposed the virtual task scheduling algorithm for reducing the makespan with the reliability constraint in [44]. Recently, Hassan *et al.* scheduled workflow with deadline constraints in the DVFS-enabled cloud environment for minimizing energy consumption [45]. Zhang *et al.* considered a bi-objective scheduling problem for the workflow energy consumption and workflow reliability in cloud computing systems [9]. Xie *et al.* [24] introduced an energy-efficient scheduling algorithm with fault-tolerant technologies for reliable applications on heterogeneous distributed embedded systems. However, most of these studies used the replication strategy for satisfying the workflow reliability constraint, which needs a large number of cloud resources for executing the repetitive tasks and results lots of energy consumption.

In addition, Tan *et al.* introduced the design of business and scientific workflows under a services computing paradigm and explained the Web services technology and workflow technology [46]. Zhu *et al.* proposed a novel scheduling method which is called matching and multi-round allocation (MMA) to optimize the makespan and total cost for all submitted tasks subject to security and reliability constraints in a heterogeneous multi-cloud environment [47]. Li *et al.* [48] considered the time-varying performance and designed a time-series-based approache to capture dynamic performance fluctuations. This approach aims at minimizing the execution cost of workflow deployed on Infrastructure-as-a-Service clouds while satisfying Service-Level-Agreements with users. These works are related with presented work. However, our work is different from the related studies [47], [48] in several aspects: (i) Tasks scheduling are considered in [47] and a scheduling problem for the workflow which usually consists of a large number of precedence constrained tasks which are connected by controlflow dependencies and data-flow is studied in this paper. (ii) [47] aims to optimize the makespan and total cost for tasks subject to security and reliability constraints and the algorithm in [48] is adopted to minimize the execution cost of workflow while satisfying Service-Level-Agreements with users. These two algorithms can not minimize scheduling energy consumption.

In this paper, we investigate a workflow scheduling problem for minimizing energy consumption and satisfying the reliability constraint in clouds. A workflow scheduling algorithm named REWS is proposed, which tries to obtain the optimal energy consumption while meeting the workflow reliability constraint. In REWS, a new sub-reliability constraint prediction strategy is proposed to break down the workflow reliability constraint to multiple task sub-reliability constraints and an update method is adopted to adjust the task sub-reliability constraint.

TABLE I
SYMBOL AND NOTATION

Symbol	Notation
$\mathcal{I}$	A set of VMs
M	The number of VMs
$\mathcal{T}$	A set of workflow tasks
N	The number of workflow tasks
$PV(t_j)$	Priority value of the task t_j
λ_m	Failure rate of the VM I_m
P_s	Static power
P_{ind}	Independent frequency dynamic power
$P_{m,ind}$	Independent frequency dynamic power of I_m
$P_{m,d}$	Frequency dependent dynamic power of I_m
$C_{m,ef}$	Effective switching capacity of VM I_m
μ_m	Dynamic power exponent of the VM I_m
K	The number of types of VMs frequency levels
$R(G)$	Reliability of the workflow
$E(G)$	Workflow energy consumption
$R_{min}(G)$	Workflow minimum reliability
$R_{max}(G)$	Workflow maximum reliability
$R_{min}(t_j)$	Minimum reliability of task t_j
$R_{max}(t_j)$	Maximum reliability of task t_j
$R_{con}(G)$	Workflow reliability constraint
$R_{pre}(t_j)$	Pre-reliability constraint of task t_j
$R_{sub}(t_j)$	Task sub-reliability constraint of task t_j
$R(t_j)$	Actual reliability of task t_j
$R(t_j, I_m, f_{m,v})$	Reliability of task t_j on I_m with $f_{m,v}$
$E(t_j)$	Actual energy of task t_j
$E(t_j, I_m, f_{m,v})$	Energy of task t_j on I_m with $f_{m,v}$
$f_{m,max}$	Maximum frequency of I_m
$f_{m,min}$	Minimum frequency of I_m
f_{m_j,v_j}	The frequency of I_{m_j} for executing task t_j
I_{m_j}	The VM for executing task t_j
$\omega_{i,m}$	Execution time of task t_i on I_m with $f_{m,max}$
$c_{i,j}$	Transfer time between task t_i and task t_j

III. MODELING AND PROBLEM FORMULATION

Next, we will introduce the system models and formulate the workflow scheduling problem as a constrained optimization problem for minimizing the energy consumption. Some important symbols used in this paper are shown in Table I.

A. Resource Model

A cloud service platform provides different cloud resources, such as CPU, RAM, disk storage and bandwidth to deal with different applications based on their requirements [49], [50]. These cloud resources are elastically used to process computing applications with different types of VMs [3], [4]. This paper considers a DVFS-enabled cloud system that consists of a set of M heterogeneous VMs denoted by $\mathcal{I} = \{I_1, I_2, \ldots, I_M\}$, where I_m represents the m-th VM instance. DVFS reduces energy consumption by lowering frequency for VMs. Each VM I_m in $\mathcal{I}$ has K types of frequency levels $(f_{m,1}, f_{m,2}, \ldots, f_{m,K})$, where $f_{m,v}$ denotes vth type frequency of VM I_m [51], [52]. VMs with the higher frequency can support the higher reliability and higher energy consumption for executing workflow.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

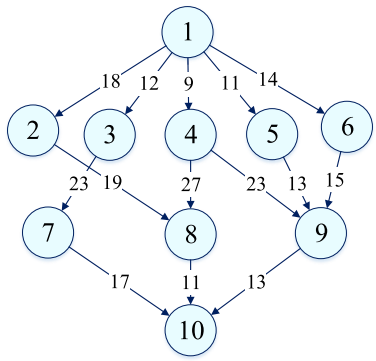


Fig. 1. A simple workflow with ten tasks.

B. Workflow Model

In general, a workflow application is modeled in the form of a directed acyclic graph (DAG) $G = (\mathcal{T}, \mathcal{E})$, where $\mathcal{T} = \{t_1, t_1, \ldots, t_N\}$ is the set of tasks and $\mathcal{E} = \{e_{ij} : i, j = 1, \ldots, N\}$ is the dependency edge set for connecting tasks. For any two tasks (t_i, t_j) in $\mathcal{T}$, the task execution order should meet the predecessor-successor relationship in workflow G. For example, the edge e_{ij} denotes that the task t_j can start after the task t_i finishing and receiving the transfer data from t_i. The task t_i is called a predecessor of t_j and task t_j is a successor of task t_i. A sample workflow with ten tasks is shown in Fig. 1. Each solid circle represents a task. The connection between tasks represents the data transmission relationship between tasks. The number on the connection represents the size of the transmission data. For example, the number 18 represents the data size of task 1 transmitted to the task 2 is 18MB.

C. Reliability Model

The transient failure and the permanent failure are two common failures for modeling reliability in clouds. Once a permanent failure occurs, the VM can only be replaced. The transient failure is the most probable occurrence, which appears for a short time and then disappears without damage to the VMs [53]. In this paper, we consider the transient failure of VMs for the workflow scheduling problem.

In general, in the reliability of the task t_j executed on the VM I_m with transient failure are calculated as

$$R(t_j, I_m) = e^{-\lambda_m w_{j,m}}. \tag{1}$$

where $w_{j,m}$ represents the execution time of task t_j on the VM I_m, and λ_m is the failure rate of I_m. According to previous research summaries, different frequencies lead to different failure rates in DVFS-based systems [24], [32]. We use $\lambda_{m,max}$ to represent the maximum frequency failure rate of VM I_m. Similar to [32], the failure rate $\lambda_{m,v}$ is calculated as

$$\lambda_{m,v} = \lambda_{m,max} \times 10^{\frac{(f_{m,max} - f_{m,v})}{(f_{m,max} - f_{m,min})}}. \tag{2}$$

where the $f_{m,v}$ represents the frequency of VM I_m, $f_{m,max}$ is the maximum frequency, and $f_{m,min}$ is the minimum frequency. According to Eq. (1) and Eq. (2), the reliability of task t_j is defined as $R(t_j, I_m, f_{m,v})$ for I_m with the frequency $f_{m,v}$. Similar to [24], we calculate it as

$$R(t_j, I_m, f_{m,v}) = e^{\frac{-\lambda_{m,v} \times (w_{j,m} \times f_{m,max})}{f_{m,v}}}. \tag{3}$$

Moreover, the workflow reliability $R(G)$ is denoted as

$$R(G) = \prod_{j=1}^{N} R(t_j) = \prod_{j=1}^{N} R(t_j, I_{m_j}, f_{m_j,v_j}). \tag{4}$$

where N is the number of tasks, $R(t_j)$ represents the actual reliability of t_j, and the reliability of t_j executed on I_{m_j} with frequency f_{m_j,v_j} is denoted as $R(t_j, I_{m_j}, f_{m_j,v_j})$.

D. Energy Model

In DVFS-capable system, it is a common way to change voltage and frequency for reducing the energy consumption of VMs [32]. The power with frequency f is defined as

$$P(f) = P_s + h(P_{ind} + P_d) = P_s + h(P_{ind} + C_{ef} f^{\mu}). \tag{5}$$

where P_s represents the static power, P_{ind} and P_d are the dynamic power. In addition, h is system states and $h = 1$ or $h = 0$. The system is active with $h = 1$. Otherwise, $h = 0$. The C_{ef} is the VM-dependent constant effective switching capacity and μ is the VM-dependent constant dynamic power exponent, which should be greater than or equal to 2.

Similar to [24], only the dynamic power P_{ind} and P_d are considered and the static power P_s are ignored in this study. A minimum energy-efficient frequency f_{ee} exists and is denoted as

$$f_{ee} = \sqrt[\mu]{\frac{P_{ind}}{(\mu - 1)C_{ef}}}. \tag{6}$$

The lowest frequency for executing a task is defined as

$$f_{low} = max(f_{min}, f_{ee}). \tag{7}$$

For the task t_j, we define the energy consumption of VM I_m with frequency $f_{m,v}$ as follows:

$$E(t_j, I_m, f_{m,v}) = (P_{m,ind} + C_{m,ef} f_{m,v}^{\mu_m}) \frac{w_{j,m} f_{m,max}}{f_{m,v}}. \tag{8}$$

where $C_{m,ef}$ is the effective switching capacity of VM I_m and μ_m is the dynamic power exponent of VM I_m. The workflow energy consumption is calculated by

$$E(G) = \sum_{t_j \in \mathcal{T}} E(t_j) = \sum_{t_j \in \mathcal{T}} E(t_j, I_{m_j}, f_{m_j,v_j}). \tag{9}$$

In this study, the spending of the frequency adjustment is very short and ignored.

E. Problem Formulation

For any task t_j, the minimum task reliability and maximum task reliability can be defined as:

$$R_{min}(t_j) = \min_{I_m \in \mathcal{I}} R(t_j, I_m, f_{m,low}), \tag{10}$$

and

$$R_{max}(t_j) = \max_{I_m \in \mathcal{I}} R(t_j, I_m, f_{m,max}), \tag{11}$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

respectively. According to Eq. (10) and Eq. (11), the minimum workflow reliability and maximum workflow reliability are defined by

$$R_{min}(G) = \prod_{t_j \in \mathcal{T}} R_{min}(t_j), \tag{12}$$

and

$$R_{max}(G) = \prod_{t_j \in \mathcal{T}} R_{max}(t_j). \tag{13}$$

respectively.

Next, we formulate the reliable workflow scheduling problem as an optimization problem as

$$\min_{\mathbf{x}} E(G) = \sum_{\upsilon=1}^{K} \sum_{m=1}^{M} \sum_{j=1}^{N} x_{j,m,\upsilon} E(t_j, I_m, f_{m,\upsilon}), \tag{14}$$

subject to

$$R_{min}(G) \leq R_{con}(G) \leq R_{max}(G), \tag{15}$$

$$R(G) \geq R_{con}(G), \tag{16}$$

$$x_{j,m,u} \leq x_{i,n,\upsilon}, \forall e_{ij} \in \mathcal{E}, m, n = 1, \ldots, M, u, \upsilon = 1, \ldots, K, \tag{17}$$

$$\sum_{m=1}^{M} \sum_{\upsilon=1}^{K} x_{j,m,\upsilon} = 1, j =, 1, \ldots, N, \tag{18}$$

$$x_{j,m,\upsilon} = \begin{cases} 1, \text{if } t_j \text{ is scheduled on } I_m \text{ with } f_{m,\upsilon}; \\ 0, \text{otherwise.} \end{cases} \tag{19}$$

The scheduling problem described by (16) is an NP-hard problem. Constraints (17) and (18) guarantee that the reliability constraint is reasonable. Constraint (19) ensures tasks execution order of workflow based on the data dependencies. Furthermore, constraints (20) and (21) ensure that every task should be scheduled and only scheduled once.

Hence, in the following sections we only consider workflow reliability constraint $R_{con}(G)$ which satisfies the following condition:

$$R_{min}(G) \leq R_{con}(G) \leq R_{max}(G). \tag{20}$$

Then, the workflow reliability $R(G)$ should satisfy that

$$R(G) = \prod_{t_j \in \mathcal{T}} R(t_j) \geq R_{con}(G). \tag{21}$$

IV. Proposed Scheduling Algorithm

In this section, we present the details of reliability-aware and energy-efficient workflow scheduling algorithm (REWS) in the Section IV. A and discuss the time complexity of REWS in the Section IV. B. Furthermore, a brief system framework which can support the algorithm implementation is introduced in Section IV. C.

A. Reliability-Aware and Energy-Efficient Workflow Scheduling Algorithm

The proposed algorithm consists of two stages: workflow pre-processing and workflow scheduling. In the workflow pre-processing stage, we calculate the task priority by the task transmission time $c_{i,j}$ and average execution time $\bar{\omega}_j$ in line 1-3 of Algorithm 1. Then, we calculate the maximum reliability and obtain pre-reliability constraint of each task in line 4-6 of Algorithm 1. Finally, we sort tasks in a list $TaskList$ by descending order of priority values and calculate the sub-reliability constraint $R_{sub}(t_1)$. The details of workflow pre-processing are shown in Algorithm 1. Moreover, the workflow scheduling stage is shown in Algorithm 2. In workflow scheduling stage, the task sub-reliability constraint is determined and one VM instance with minimum energy consumption is selected to schedule the task for satisfying it's sub-reliability constraint. Firstly, we calculate the task reliability of the entry task in lines 3-4 of Algorithm 2. Then, the sub-reliability constraint of the other tasks are obtained with an update strategy in lines 5-11. The total reliability of assigned tasks compares with the $(j-1)$-th power of the mean square value of the workflow constraint $\sqrt[N]{R_{con}(G)})^{j-1}$ for the jth task and we obtain the task sub-reliability constraint for the task t_j in lines 10-11. Based on the above information, a VM is heuristically allocated to the task for achieving minimum energy consumption while satisfying the workflow reliability constraint in lines 12-27 of Algorithm 2.

Algorithm 1 Workflow Pre-Processing Algorithm

Input: The task set $\mathcal{T}$ of workflow G, a VMs set $\mathcal{I}$ and the worklfow reliability constraint $R_{con}(G)$

Output: The priority value and the pre-reliability constraint of each task in $\mathcal{T}$ and the sub-reliability constraint $R_{sub}(t_1)$ of task t_1.

1: **for** each task $t_j \in \mathcal{T}$ **do**
2: Calculate the average execution time $\bar{\omega}_j$ of task t_k by Eq. (22).
3: Calculate the $priority$ values of task t_k using Eq. (23).
4: **for** each task t_k in $\mathcal{T}$ **do**
5: Calculate the maximum reliability $R_{max}(t_k)$ using Eq.(11).
6: Calculate the pre-reliability $R_{pre}(t_k)$ using Eq. (24).
7: Sort tasks in a list $TaskList$ by descending order of priority values.
8: **return** Result

Next, we will explain the proposed approach in detail with four steps for workflow pre-processing and workflow scheduling, including task prioritization, sub-reliability constraint prediction, sub-reliability constraint assignment and task scheduling.

1) Task Prioritization: At first, we calculate the average execution time $\bar{\omega}_j$ of the task t_j as

$$\bar{\omega}_j = \frac{\sum_{m=1}^{M} \omega_{j,m}}{M}, \tag{22}$$

where $\omega_{j,m}$ represents the execution time of task t_j with VM instance I_m. Then, we use the $\bar{\omega}_j$ to obtain the scheduling priority of task t_j in the workflow pre-processing stage. The task scheduling priority value $PV(t_j)$ is denoted as

$$PV(t_j) = \bar{\omega}_j + \max_{t_i \in succ(t_j)} \{c_{j,i} + PV(t_i)\}. \tag{23}$$

where $c_{j,i}$ is data transmission time from the VM executing task t_j to the VM executing t_i, and the immediate successors

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Reliability-Aware and Energy-Efficient Workflow Scheduling Algorithm

Input: A task sequence $TaskList$ by descending order of priority values and a VMs set $\mathcal{I} = \{I_1, I_2, \ldots, I_M\}$.
Output: The scheduling solution of task t_j.

```
(I*, f_{m*,v*}) = null, E(t_j) = ∞, AssignedTaskset = null, R_ass(G) = 1.
for each task t_j ∈ T do;
    if t_j == t_1 then
        R_sub(t_j) = R_sub(t_1);
    else
        Calculate the sub-reliability constraint R_sub(t_j) of task t_j by Eq. (43);
        for each task t_k in AssignedTaskset do
            Calculate the reliability R(t_k) using Eq.(3);
        Calculate the reliability R^j_ass(G) using Eq.(26);
        if R^j_ass > (N√R_con(G))^{j-1} then
            R_sub(t_j) = min{R_sub(t_j), N√R_con(G)};
for each VM I_m ∈ I do
    for each frequency f_{m,v} ∈ [f_{m,low}, f_{m,max}] do
        Calculate R(t_j, I_m, f_{m,v}) using Eq. (3);
        if R(t_j, I_m, f_{m,v}) > R_sub(t_j) then
            Calculate E(t_j, I_m, f_{m,v}) for the task t_j by Eq. (9);
            if E(t_j, I_m, f_{m,v}) < E(t_j) then
                E(t_j) ← E(t_j, I_m, f_{m,v});
                (I*, f_{m*,v*}) ← (I_m, f_{m,v});
                R(t_j) ← R(t_i, I_m, f_{m,v});
if (I*, f_{m*,v*}) = null then
    Calculate maximal reliability R_max(t_j) by Eq. (11).
    R(t_j) = R_max(t_i)
    (I*, f_{m*,v*}) = (I_m, f_{m,v}) with R_max(t_j) for the task t_j;
    Calculate E(t_j, I*, f_{m*,max}) for the task t_j by Eq. (9);
    E(t_j) = E(t_j, I*, f_{m*,max});
    Add the task t_j into AssignedTaskset;
return Result
```

of task t_j is denoted as $succ(t_j)$. Specially, for the leaf-node task t_j, we calculate the priority of t_j with $PV(t_j) = \bar{\omega}_j$.

2) Sub-Reliability Constraint Prediction: We sort these tasks as a sequence $\{t_1, \ldots, t_j, t_{j+1}, \ldots, t_N\}$ with the task scheduling priority. Then, a sub-reliability constraint prediction strategy is designed to break down the workflow reliability constraint and assign a pre-reliability constraint $R_{pre}(t_j)$ for task t_j to guarantee the workflow reliability constraint. By this strategy, the pre-reliability constraint $R_{pre}(t_j)$ of task t_j is calculated as

$$R_{pre}(t_j) = R_{max}(t_j)\sqrt[N]{\frac{R_{con}(G)}{R_{max}(G)}}. \tag{24}$$

If t_j is a scheduling task, $\{t_1, \ldots, t_{j-1)}\}$ is the scheduled task set and $\{t_{j+1}, \ldots, t_N\}$ is unscheduled. Then, the workflow reliability $R_j(G)$ with the task t_j is calculated by

$$R_j(G) = R^j_{ass}(G)R(t_j)R^j_{pre}(G), \tag{25}$$

where $R^j_{ass}(G)$ and $R^j_{pre}(G)$ represent the assigned reliability for scheduled tasks and the total pre-reliability for unscheduled tasks, respectively. We calculate them as

$$R^j_{ass}(G) = \prod_{y=1}^{j-1} R(t_y), \tag{26}$$

and

$$R^j_{pre}(G) = \prod_{y=j+1}^{N} R_{pre}(t_y). \tag{27}$$

To illustrate the effectiveness of the sub-reliability constraint prediction strategy, we prove that the workflow reliability $R_j(G)$ will satisfy the reliability constraint $R_{con}(G)$ for any task t_j with Theorem 1.

Theorem 1: Let $t_j (j = 1, \ldots, N)$ be a task of the workflow G and $R(t_j)$ denote the actual reliability of task t_j, there exits a VM I_{m_j} with a frequency f_{m_j,v_j} such that

$$R(t_j) = R(t_j, I_{m_j}, f_{m_j,v_j}), \tag{28}$$

and

$$R_j(G) = R^j_{ass}(G)R(t_j)R^j_{pre}(G) \geq R_{con}(G),\ j = 1, \ldots, N, \tag{29}$$

where $R^j_{ass}(G)$ and $R^j_{pre}(G)$ are defined by Eq. (26) and Eq. (27), respectively.

Proof: For the entry task t_1, if

$$R_1(G) = R(t_1)R^1_{pre}(G) \geq R_{con}(G), \tag{30}$$

then

$$R(t_1) \geq \frac{R_{con}(G)}{R^1_{pre}(G)}. \tag{31}$$

That is, we should prove that there exits a VM I_{m_1} with a frequency f_{m_1,v_1} such that

$$R(t_1, I_{m_1}, f_{m_1,v_1}) = R(t_1) \geq \frac{R_{con}(G)}{\prod_{y=2}^{N} R_{pre}(t_y)}. \tag{32}$$

and

$$R_{min}(t_1) \leq R(t_1) \leq R_{max}(t_1). \tag{33}$$

Since

$$\frac{R_{con}(G)}{\prod_{y=2}^{N} R_{pre}(t_y)} = \frac{R_{con}(G)}{\prod_{y=2}^{N}(R_{max}(t_y)\sqrt[N]{\frac{R_{con}(G)}{R_{max}(G)}})} \leq R_{max}(t_1). \tag{34}$$

Therefore, t_1 is able to find an assigned VM to satisfy Eq. (28), namely

$$R_{max}(t_1) \geq R(t_1, I_{m_1}, f_{m_1,v_1}) = R(t_1) \geq \frac{R_{con}(G)}{R^1_{pre}(G)}. \tag{35}$$

Then, we obtain

$$R_1(G) = R(t_1, I_{m_1}, f_{m_1,v_1})R^1_{pre}(G) \geq R_{con}(G). \tag{36}$$

For the task $t_j (j > 1)$, if we can find an assigned VM I_{m_j} with a frequency f_{m_j,v_j} to satisfy the $R_{con}(G)$, we have

$$\begin{aligned} R_j(G) &= R^j_{ass}(G)R(t_j, I_{m_j}, f_{m_j,c_j})R^j_{pre}(G) \\ &= R^{j+1}_{ass}(G)R^j_{pre}(G) \\ &\geq R_{con}(G), \end{aligned} \tag{37}$$

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

Hence, it should have

$$R_{ass}^{j+1}(G) \geq \frac{R_{con}(G)}{R_{pre}^{j}(G)}. \tag{38}$$

For the task t_{j+1}, we have

$$R_{j+1}(G) = R_{ass}^{j+1}(G)R(t_{j+1})R_{pre}^{j+1}(G). \tag{39}$$

By the Eq. (38), we obtain

$$\begin{aligned} R_{j+1}(G) &\geq \frac{R_{con}(G)}{R_{pre}^{j}(G)}R(t_{j+1})R_{pre}^{j+1}(G) \\ &= R_{con}(G)\frac{R(t_{j+1})}{R_{pre}(t_{j+1})}. \end{aligned} \tag{40}$$

In addition, when $R(t_{j+1}) = R_{max}(t_{j+1})$, we have

$$R_{j+1}(G) \geq R_{con}(G). \tag{41}$$

That is, we can also find an assigned VM for t_{j+1} to satisfy the reliability constraint $R_{con}(G)$. □

3) Sub-Reliability Constraint Assignment: In this subsection, we determine the sub-reliability constraint for each task. Firstly, based on Theorem 1, we obtain

$$R(t_j) \geq \frac{R_{con}(G)}{R_{ass}^{j}(G)R_{pre}^{j}(G)}. \tag{42}$$

Then, we can set the sub-reliability constraint of task t_j as

$$R_{sub}(t_j) = \frac{R_{con}(G)}{R_{ass}^{j}(G)R_{pre}^{j}(G)}. \tag{43}$$

In this way, the workflow reliability constraint is satisfied with the task sub-reliability constraints. That is, the actual task reliability $R(t_j)$ for task t_j must satisfy

$$R(t_j) \geq R_{sub}(t_j). \tag{44}$$

Furthermore, the sub-reliability constraint of unassigned tasks are adjusted and determined in lines 3-11 of Algorithm 2.

For a workflow, it is obvious that the workflow constraint can be met when all of task sub-reliability constraints are set as $\sqrt[N]{R_{con}(G)}$. In this way, we design a update strategy to adjust the task sub-reliability constraint which is obtained by Eq. (43). Based on the proposed update strategy, the task sub-reliability constraint can be adjusted while all of its parents tasks have been completed. Then, for task t_j, the sub-reliability constraint can be calculated as

$$R_{sub}(t_j) = \begin{cases} R_{sub}(t_j), \text{if } R_{ass}^{j} \leq (\sqrt[N]{R_{con}(G)})^{j-1}; \\ \min\{R_{sub}(t_j), \sqrt[N]{R_{con}(G)}\}, otherwise. \end{cases} \tag{45}$$

4) Task Scheduling: For any task t_i, the main idea of task scheduling is to schedule task t_i to one VM instance, which can minimize its energy consumption. We explain the task scheduling in lines 12-27 of Algorithm 2 and the detailed processes are as follows.

- A task t_j with the highest priority is selected from the task queue $TaskList$. We compute the task reliability $R(t_j, I_m, f_{m,v})$ with VM I_m for any $I_m \in \mathcal{I}$ by Eq. (3) and then obtain the sub-reliability constraint $R_{sub}(t_j)$ for task t_j by Eq. (45).
- We schedule task t_j to satisfy its sub-reliability constraint on VM instances. If task t_j can be scheduled to multiple VM instances, we calculate the energy consumption of task t_j on each I_m in $\mathcal{I}$ and choose one VM instance I^* with minimal energy consumption for executing task t_j. If there is no VM instance to meet the sub-reliability constraint for task t_j, we choose one VM instance with maximal reliability for executing task t_j.
- For task t_j, if it has been scheduled to VM instances, we remove it from the task queue $TaskList$. The task scheduling sequence and the sub-reliability constraint of unscheduled tasks are updated. Then, next unscheduled task with the highest priority is scheduled to VM instances until all tasks have been finished.

B. Time Complexity Analysis

The time complexity of the proposed approach is analyzed in this section. Our approach consists of two stages: workflow pre-processing (Algorithm 1) and workflow scheduling (Algorithm 2). Algorithm 1 mainly contains two parts, i.e., task prioritization and sub-reliability constraint prediction, where the complexity of task prioritization is determined by lines 2-3, it has the complexity of $O(N)$ to compute the task priority values Then, the the complexity of sorting the tasks with descending order of priority values in line 7 is $O(NlogN)$, where N represents the number of tasks for workflow. In the sub-reliability constraint prediction part, it has the complexity of $O(N)$ to calculate the maximal sub-reliability constraint and pre-reliability constraint of tasks in lines 4-6. Hence, the complexity of Algorithm 1 is $2O(N) + O(NlogN)$. In Algorithm 2, it has the complexity of $O(N^2)$ to obtain the task sub-reliability constraint in lines 2-11. Then, the task scheduling part is mainly obtained by lines 12-27 and has the complexity of $O(NMK)$, where M and K denote the number of VMs instances and frequency for each VM instance, respectively. Therefore, Algorithm 2 has the complexity of $O(N^2) + O(NMK)$.

C. Reliability-Aware and Energy-Efficient Workflow Scheduling Framework

To support our algorithm implementation, we build a reliability-aware and energy-efficient workflow scheduling framework. As shown in Fig. 2, the framework consists of five components, including workflow analyzer, reliability decomposer, resource manager, workflow scheduler and feedback processer. Specifically, workflow analyzer is responsible for obtaining the task priority for the workflow with the reliability requirement. Reliability decomposer transforms the workflow reliability constraint to task sub-reliability constraints with the sub-reliability constraint prediction strategy. Resources manager scales out VMs from IaaS clouds and obtains the VMs set which meets the task sub-reliability constraint for the task. Workflow scheduler selects a VM with the minimum energy consumption to the task for meeting the task sub-reliability constraint. Feedback processer updates the unscheduled task sub-reliability constraint according to the assigned

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

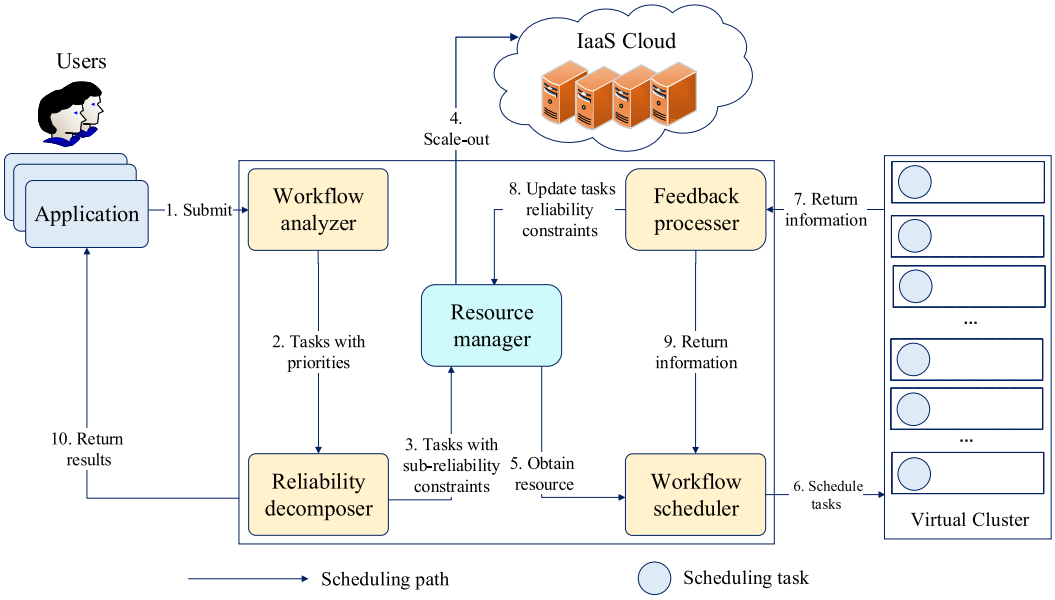


Fig. 2. Reliability-aware and energy-efficient workflow scheduling framework.

TABLE II
SOME VM PARAMETERS ARE USED IN EXPERIMENTS

$w_{j,m}$	$10ms \leq w_{j,m} \leq 100$ ms
$P_{m,ind}$	$0.03 \leq P_{m,ind} \leq 0.07$
$C_{m,ef}$	$0.8 \leq C_{m,ef} \leq 1.2$
μ_m	$2.5 \leq \mu_m \leq 3.0$

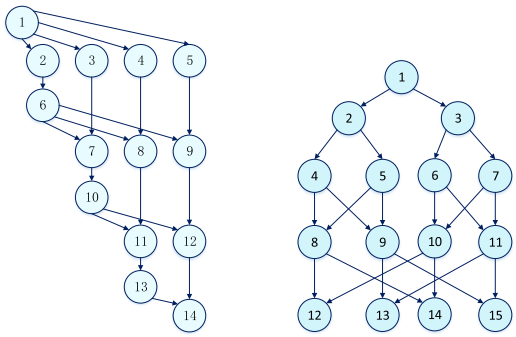
reliability with the as much as possible for reducing the energy consumption.

V. PERFORMANCE EVALUATION

A. Experimental Environment

1) Experimental Setting: Experimental environment includes CPU (Intel quad core, 3.4 GHz), memory (16.0 GB), Windows 10 operating system, Eclipse, JDK7.0. Some VMs with different computing abilities are used to be a simulated cloud computing platform for our experiments. The VM parameters are used as same as [24] and shown in Table II. The values of these parameters are generated by uniform distribution. The frequency of VMs are discrete with 0.01 GHz resolution and maximum frequency are 1.0 GHz.

We used three different types of workflow, Gaussian elimination, fast Fourier transform and randomly generated workflow applications [54], to evaluate our algorithm in the experiments. The parallelism of task for Gaussian elimination workflow and fast Fourier transform workflow is different, where the gaussian elimination is with the low task parallelism and the fast fourier transform is high. Similar to [54], a parameter ρ is used to denote the size of the Gaussian elimination workflow and the fast Fourier transform workflow. With the parameter ρ, the number of tasks for the Gaussian elimination workflow and the fast Fourier transform workflow can be calculated as $N = \frac{\rho^2+\rho-2}{2}$ and $N = (2\rho-1)+\rho\log_2\rho$, respectively. Specially, the parameter ρ for the fast Fourier transform workflow should satisfy that $\rho = 2^y$ for some



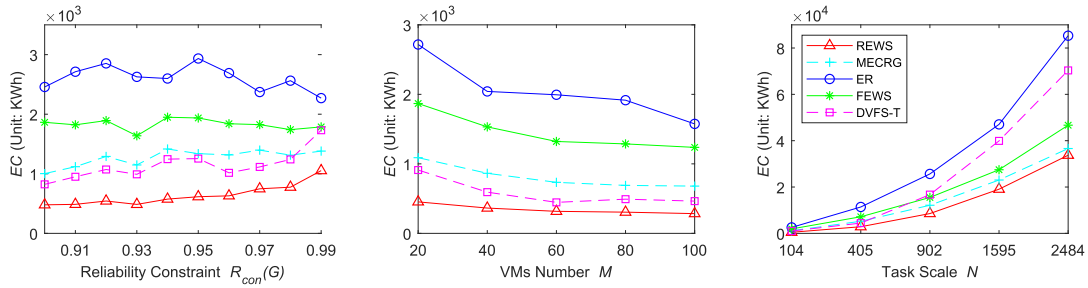
(a) The Gaussian elimination workflow with $\rho = 5$. (b) The fast Fourier transform workflow with $\rho = 4$.

Fig. 3. Examples of real-world workflow applications.

integer y. The example of the Gaussian elimination workflow with $\rho = 5$ is shown in Fig. 3(a). Fig. 3(b) shows an example of the fast Fourier transform workflow with $\rho = 4$. For randomly generated workflow, one uses the task graphs for free 3.5 to generate the random workflow [55].

2) Comparison Algorithms: The reliability and resource (RR) algorithm [42], the minimizing resource consumption cost with reliability goal (MRCRG) algorithm [54], the dynamic voltage frequency scaling technique (DVFS-T) algorithm [56] and the fault-tolerant cost-efficient wokflow scheduling algorithm (FCWS) [57] are used to schedule the workflow application for satisfying the workflow reliability constraint. We extend RR to optimize energy consumption and meet the reliability requirement (ER) for comparing with REWS. In [24], MRCRG is extended to minimizing energy consumption with reliability goal (MECRG) for scheduling reliable parallel applications on heterogeneous distributed embedded systems. In addition, FCWS is extended to fault-tolerant energy-efficient wokflow scheduling algorithm (FEWS) which is

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.



(a) N=104, M=20. (b) N=104, $R_{con}(G)$=0.90. (c) M=20, $R_{con}(G)$=0.90.

Fig. 4. The EC of scheduling gaussian elimination workflow.

designed to optimize energy consumption and satisfy the workflow reliability constraint in clouds. DVFS-T algorithm is an energy-minimized workflow scheduling approach for real-time parallel workflows with the reliability constraint. Thus, the baselines are as follows:

ER: The task sub-reliability constraint of the highest priority task can be calculated as

$$R_{sub}(t_1) = \sqrt[N]{R_{con}(G)}. \tag{46}$$

We use the the maximum reliability value of unscheduled tasks to obtain the the task sub- reliability constraint. For the rest of tasks, the task sub-reliability constraint is calculated by

$$R_{sub}(t_j) = \sqrt[N-j+1]{\frac{R_{con}(G)}{R_{max}(G)}}, j = 2, \ldots, N. \tag{47}$$

Then, with the ER algorithm, we select the VM with minimum energy consumption to execute task t_j while satisfying the the task sub-reliability constraint $R_{sub}(t_j)$.

MECRG: The task sub-reliability constraint of task t_j in MECRG can be calculated as

$$R_{sub}(t_j) = \frac{R_{con}(G)}{\prod_{y=1}^{j-1} R(t_y) \prod_{y=j+1}^{N} R_{max}(t_y)}. \tag{48}$$

Then, each task is assigned to a VM with the minimum energy consumption while meeting its reliability constraint.

DVFS-T: The DVFS-T algorithm consists of two stage: task pre-scheduling with minimizing schedule length (MSL) method and task scheduling with minimizing energy consumption. Firstly, the task sub-reliability constraint $R_{sub}^{MSL}(t_j)$ for task t_j can be calculated wit Eq. (48) and t_j is assign to VMs with minimizing schedule length in the task pre-scheduling stage. Based on the schedule from MSL, the obtained workflow reliability $R^{MSL}(G)$ is verified to be greater than $R_{con}(G)$. Then, we define the reliability ratio β_r as

$$\beta_r = \frac{\sqrt[N]{R_{con}(G)}}{\sqrt[N]{R^{MSL}(G)}}, \tag{49}$$

where $R^{MSL}(G)$ is calculated based on the schedule from MSL. Next, task t_j is reallocated in the task scheduling stage and the task sub-reliability constraint on this task is

$$R_{sub}(t_j) = \frac{R_{con}(G)}{(\beta_r)^j \prod_{y=1}^{j-1} R(t_y) \prod_{y=j+1}^{N} R^{MSL}(t_y)}, \tag{50}$$

where $R^{MSL}(t_y)$ denotes the task reliability for t_y with MSL in the task pre-scheduling stage. Finally, each task is assigned to a VM with the minimum energy consumption while meeting the task sub-reliability constraint $R_{sub}(t_j)$.

FEWS: The FEWS is designed to optimize energy consumption and satisfy the workflow reliability constraint for scheduling workflows in clouds. In FEWS, the task is assigned to a VM with a maximum energy and reliability aware scheduling metric $SM(t_j, I_m, f_{m,v})$, which is defined as

$$SM(t_j, I_m, f_{m,v}) = \alpha \frac{E(t_j, I_m, f_{m,v}) - E_{min}(t_j)}{E_{max}(t_j) - E_{min}(t_j)} + \beta \frac{R(t_j, I_m, f_{m,v}) - R_{min}(t_j)}{R_{max}(t_j) - R_{min}(t_j)}, \tag{51}$$

where $\alpha + \beta = 1$.

3) Performance Metrics: To evaluate the performance of methods, we use the following metrics: Energy Consumption (EC) and Normalized Reliability (NR). Each metric is demonstrated as follows.

Energy Consumption (EC): We calculate the total cost with all active VMs as

$$EC = \sum_{i=1}^{N} E(t_i), \tag{52}$$

where $E(t_i)$ represents the energy consumption of task t_i.

Normalized Reliability (NR): We use the normalized reliability of workflow to evaluate the three algorithms, which is defined as

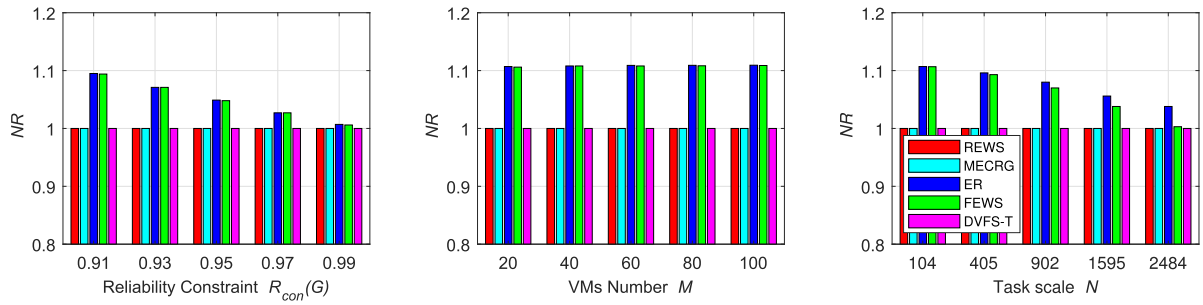
$$NR = \frac{R(G)}{R_{con}(G)}, \tag{53}$$

where $R(G)$ and $R_{con}(G)$ represent the actual workflow reliability and the workflow reliability constraint, respectively. The workflow reliability constraint is not satisfied with $NR < 1$. Otherwise, $NR \geq 1$.

B. Experimental Results

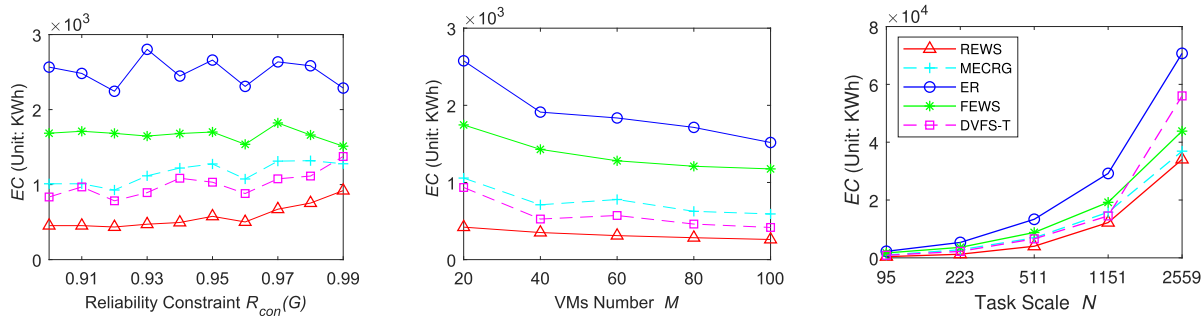
In this section, we introduce experiments of three different workflows to evolute the five algorithms. The experimental results are shown in Fig. 4-Fig. 9. The results of EC and NR for for the Gaussian elimination workflow are shown in Fig. 4 and Fig. 5, respectively. Fig. 6 and Fig.7 show the results of EC and NR of fast Fourier transform workflow. Moreover, the experimental results for average EC and average NR of

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.



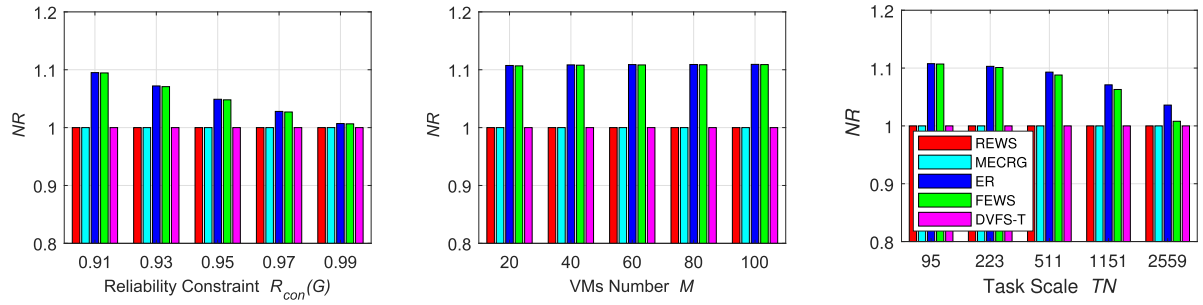
(a) N=104, M=20. (b) N=104, $R_{con}(G)$=0.90. (c) M=20, $R_{con}(G)$=0.90.

Fig. 5. The NR of scheduling gaussian elimination workflow.



(a) N=95, M=20. (b) N=95, $R_{con}(G)$=0.90. (c) M=20, $R_{con}(G)$=0.90.

Fig. 6. The EC of scheduling fast fourier transform workflow.



(a) N=95, M=20. (b) N=95, $R_{con}(G)$=0.90. (c) M=20, $R_{con}(G)$=0.90.

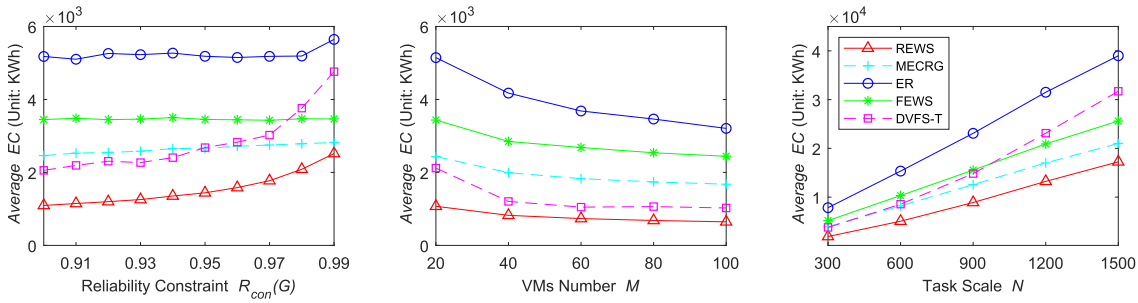
Fig. 7. The NR of scheduling fast fourier transform workflow.

50 randomly generated workflow are shown in Fig. 8 and Fig. 9, respectively.

1) Gaussian Elimination Workflow: In Fig. 4, the energy consumption EC of the Gaussian elimination workflow of REWS is least comparing with the other four algorithms in different cases. Moreover, Fig. 5 shows that the normalized reliability NR is always greater than or equal to 1, which means that the workflow reliability constraints can be achieved by five algorithms in all cases. Fig. 4(a) and Fig. 5(a) show the results for the Gaussian elimination workflow on different reliability constraints with $N = 104$ and $M = 20$. As can be seen, the EC increases with the increased workflow reliability constraints and is always the least by REWS in Fig. 4(a). Furthermore, Fig. 5(a) shows that five algorithms can satisfy the different workflow reliability constraints. Fig. 4(b) and Fig. 5(b) show the results of the EC and NR of Gaussian elimination workflow on varying number of VMs while the number of tasks $N = 104$ and the workflow reliability constraint $R_{con}(G) = 0.90$, respectively. In Fig. 4(b), the EC decreases with the increased number of VMs and is always the least by REWS in all cases. In addition, the NR is always greater than or equal to 1, which means that the workflow reliability constraint can be satisfied with the different number of VMs in Fig. 5(b). From Fig. 4(c), we can obviously see that the EC obtained by REWS is less than the other four algorithms on different task scales while the number of VMs $M = 20$ and the workflow reliability constraint $R_{con}(G) = 0.90$. Moreover, five algorithms can satisfy the workflow reliability constraint with different task scales in Fig. 5(c).

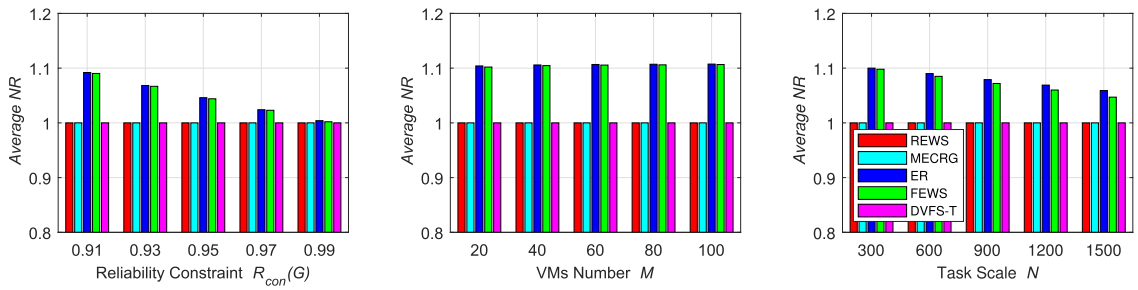
Specially, the results in Fig. 5 show that the workflow reliability with REWS is always very close to the workflow reliability with MECRG and DVFS-T, and the workflow reliability with ER is always the highest of the five algorithms. The reason is that REWS and MECRG belong to the greedy strategies for decomposing the workflow reliability constraint. Both

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.



(a) N=200, M=20. (b) N=200, $R_{con}(G)$=0.90. (c) M=20, $R_{con}(G)$=0.90.

Fig. 8. The average EC of scheduling 50 random generated workflow.



(a) N=200, M=20. (b) N=200, $R_{con}(G)$=0.90. (c) M=20, $R_{con}(G)$=0.90.

Fig. 9. The average NR of scheduling 50 random generated workflow.

REWS and MECRG make much greater use of the assigned reliability constraint for minimizing the energy consumption. As a result, the difference in reliability with REWS and MECRG is very small. But ER uses fault-tolerant technology to perform the same task repeatedly for meeting the reliability constraint, which results in hugest energy consumption. In addition, a new sub-reliability constraint prediction strategy in REWS is used to decompose the workflow reliability constraint and determine the pre-reliability constraint of each task. Moreover, the task sub-reliability constraint is adjusted by a new update method to reduce energy consumption. Therefore, REWS achieves the lowest energy consumption for workflow while satisfying the reliability constraint, as compared to the other four algorithms. The same is true for fast Fourier transform applications and randomly generated workflow applications in the following.

2) Fast Fourier Transform Workflow: In this subsection, experiments are designed with fast Fourier transform workflow for EC and NR in five algorithms. Fig.6 shows the results of EC obtained by five algorithms with the varying workflow reliability constraints, number of VMs and task scales. Moreover, the results of the NR of five algorithms on the different workflow reliability constraint, number of VMs and tasks scales, are shown in Fig. 7. Fig. 6(a) and Fig. 7(a) are the results of the EC and NR of the fast Fourier transform workflow with different workflow reliability constraints, respectively. The workflow reliability constraints is denoted as $\{0.90, 0.91, 0.92, 0.93, 0.94, 0.95, 0.96, 0.97, 0.97, 0.98, 0.99\}$. The task scale $N = 95$ and the number of VMs $M = 20$. From Fig. 8(a), the EC obtained by REWS is less than the four compared algorithms. Fig. 7(a) shows that the workflow reliability constraint is always satisfied by five algorithms. Similarly to Fig. 4(b) and Fig. 5(b), the EC decreases with the increased number of VMS and is the least with REWS compared with other four algorithms, and all of these algorithms always satisfy the workflow reliability constraint in all cases while $N = 95$ and $R_{con}(G) = 0.90$ in Fig. 6(b) and Fig. 7(b). According to Fig. 6(c) and Fig. 7(c), the EC of fast Fourier transform workflow obtained by REWS is less than four compared algorithms and all algorithms can satisfy the workflow reliability constraints on different task scales while the number of VMs $M = 20$ and the workflow reliability constraint $R(G) = 0.90$.

3) Randomly Generated Workflow: To further illustrate the effectiveness of REWS, the experimental results for average EC and average NR of 50 randomly generated workflow [24] are shown in Fig. 8 and Fig. 9, respectively. The average EC and average NR of 50 random generated workflow with $N = 200$ and $M = 20$ for the different workflow reliability constraints are shown in Fig. 8(a) and Fig. 9(a), respectively. The experiments results show that REWS algorithm is much better than the four compared algorithms for reducing energy consumption and all of these algorithms can achieve the different workflow constraints. Furthermore, Fig. 8(b) indicates that the average energy consumptions decreases when the number of VMs increases and the average energy consumptions of REWS are always the least. From Fig. 9(b), we can obtain that five algorithms can satisfy the reliability constraint 0.90 with $N = 200$ tasks for the different number of VMs. In addition, we compare the average EC and average NR of 50 randomly generated workflow with $M = 20$ and $R_{con}(G) = 0.90$ by varying the task scale N. According to Fig. 8(c), the average EC of REWS are smaller than other four algorithms on different task scales of randomly generated

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

TABLE III
RUNTIME OF FIVE ALGORITHMS

Task Scale N	Runtime (unit: ms)				
	REWS	MECRG	ER	FEWS	DVFS-T
300	305	282	92	489	3821
600	627	565	192	996	13286
900	986	855	332	1512	27556
1200	1360	1153	500	2052	30661
1500	1770	1460	711	2585	35732

workflow when $M = 20$ and $R_{con}(G) = 0.90$. Fig. 9(c) shows that all algorithms can satisfy the workflow reliability constraints on different task scales. These results confirm that REWS is more efficient than MECRG, ER, FEWS and DVFS-T algorithms in all cases. In addition, we consider that the reliability constraint $R_{con}(G)$ is between minimum reliability $R_{min}(G)$ and maximum reliability $R_{max}(G)$ in this paper. The actual reliability by REWS and MECRG can not greater than $R_{max}(G)$. If the reliability constraint $R_{con}(G)$ is greater than $R_{max}(G)$, the REWS and MECRG will fail to meet.

C. Scalability

We investigate the scalability of five algorithms by the randomly generate workflows. The task scale in randomly generate workflows $N = \{300, 600, 900, 1200, 1500\}$. The runtime of each approach is shown in Table III. From Table III, we can see that the runtime of five algorithms increase with the increasing of task scales. Specially, ER has the minimum runtime in different task scales. It uses the maximum reliability value of unscheduled tasks to obtain the task sub-reliability constraint for meeting the workflow reliability constraint, which can reduce the computational overhead, but it also brings huge energy consumption. Moreover, both REWS and MECRG calculate the total realiability of scheduled tasks and the total pre-reliability of unscheduled tasks to obtain the sub-reliability constraint for the scheduling task. Furthermore, REWS adopts an update method to adjust the task sub-reliability constraint for further reducing energy consumption. Then, the computational overhead of REWS is slightly greater than MECRG, but the energy consumption of REWS is much lower than MECRG. In addition, the runtime of FEWS is longer than REWS. The computational overhead of the DVFS-T is much greater than the other four algorithms in all cases. Based on the above analysis, comparing with the other four algorithms, REWS has the lower computational overhead and the lowest energy consumption. Hence, REWS has the good scalability and scheduling performance.

VI. CONCLUSION AND FUTURE WORK

Workflow scheduling is a challenging issue in emerging trends of the cloud environment that focuses on satisfying various QoS constraints. In this paper, we investigate a reliability-aware and energy-efficient workflow scheduling problem in IaaS clouds. A workflow scheduling algorithm named REWS is proposed to obtain the optimal energy consumption while meeting the workflow reliability constraint. The basic idea is to divide workflow reliability constraint into task sub-reliability constraints and schedule tasks with an energy-efficient scheduling strategy. Furthermore, a workflow scheduling framework which consists of workflow analyzer, reliability decomposer, resource manager, workflow scheduler and feedback processer, is designed to support the algorithm implementation. Experiments results show that REWS can always satisfy the workflow reliability constraint and is more effective compared with other four algorithms and can be suitable for different kinds of the real-world workflow application in clouds.

We discuss some possible developments of our research work which may inspire future research work. For one thing, our research work does not consider dynamic multi-workflows scheduling problem. The real case is that many computing service requests are submitted to the cloud. It is very meaningful to design the efficient scheduling algorithms to handle the dynamic applications on the fly. Based on the related work, an optimal solution is to take into account the dynamic multi-workflow scheduling problem in our future work. For another, our proposed algorithm focuses on the heuristic approaches and lacks the exploration of intelligent optimization algorithms. We plan to study the principles of some classic intelligent optimization algorithms [58]–[60] and solve our research problem based on some intelligence optimization algorithms in the future work.

REFERENCES

[1] X. Li and Z. Cai, "Elastic resource provisioning for cloud workflow applications," *IEEE Trans. Autom. Sci. Eng.*, vol. 14, no. 2, pp. 1195–1210, Apr. 2017.
[2] H. Li, B. Wang, Y. Yuan, M. Zhou, Y. Fan, and Y. Xia, "Scoring and dynamic hierarchy-based NSGA-II for multiobjective workflow scheduling in the cloud," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 2, pp. 982–993, Apr. 2022, doi: 10.1109/TASE.2021.3054501.
[3] Z. Wang, M. M. Hayat, N. Ghani, and K. B. Shaban, "Optimizing cloud-service performance: Efficient resource provisioning via optimal workload allocation," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 6, pp. 1689–1702, Jun. 2017.
[4] X. Zuo, G. Zhang, and W. Tan, "Self-adaptive learning PSO-based deadline constrained task scheduling for hybrid IaaS cloud," *IEEE Trans. Autom. Sci. Eng.*, vol. 11, no. 2, pp. 564–573, Apr. 2014.
[5] Z. Cai, X. Li, and J. N. D. Gupta, "Heuristics for provisioning services to workflows in XaaS clouds," *IEEE Trans. Serv. Comput.*, vol. 9, no. 2, pp. 250–263, Mar. 2016.
[6] X. Mei, X. Chu, H. Liu, Y.-W. Leung, and Z. Li, "Energy efficient real-time task scheduling on CPU-GPU hybrid clusters," in *Proc. IEEE INFOCOM*, May 2017, pp. 1–9.
[7] H. Yuan, J. Bi, and M. Zhou, "Energy-efficient and QoS-optimized adaptive task scheduling and management in clouds," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 2, pp. 1233–1244, Apr. 2022, doi: 10.1109/TASE.2020.3042409.
[8] K. Li, "Power and performance management for parallel computations in clouds and data centers," *J. Comput. Syst. Sci.*, vol. 82, no. 2, pp. 174–190, Mar. 2016.
[9] L. Zhang, K. Li, C. Li, and K. Li, "Bi-objective workflow scheduling of the energy consumption and reliability in heterogeneous computing systems," *Inf. Sci.*, vol. 379, pp. 241–256, Feb. 2017.
[10] *The Website of NRDC*. Accessed: Sep. 1, 2021. [Online]. Available: https://www.nrdc.org/
[11] G. Xie, G. Zeng, X. Xiao, R. Li, and K. Li, "Energy-efficient scheduling algorithms for real-time parallel applications on heterogeneous distributed embedded systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 12, pp. 3426–3442, Dec. 2017.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

[12] C.-G. Wu and L. Wang, "A multi-model estimation of distribution algorithm for energy efficient scheduling under cloud computing system," *J. Parallel Distrib. Comput.*, vol. 117, pp. 63–72, Jul. 2018.

[13] A. Dogan and F. Ozguner, "Matching and scheduling algorithms for minimizing execution time and failure probability of applications in heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 308–323, Mar. 2002.

[14] X. Li, L. Qian, and R. Ruiz, "Cloud workflow scheduling with deadlines and time slot availability," *IEEE Trans. Serv. Comput.*, vol. 11, no. 2, pp. 329–340, Mar. 2018.

[15] H. Liu, Y. Xia, L. Wu, and P. Chen, "A novel reinforcement-learning-based approach to scientific workflow scheduling," in *Proc. IEEE Int. Conf. Netw., Sens. Control (ICNSC)*, Oct. 2020, pp. 1–8.

[16] A. A. Al-Habob, O. A. Dobre, A. G. Armada, and S. Muhaidat, "Task scheduling for mobile edge computing using genetic algorithm and conflict graphs," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8805–8819, Aug. 2020.

[17] Q. Wu, M. Zhou, and J. Wen, "Endpoint communication contention-aware cloud workflow scheduling," *IEEE Trans. Autom. Sci. Eng.*, vol. 19, no. 2, pp. 1137–1150, Apr. 2022, doi: 10.1109/TASE.2020.3046673.

[18] Z. Wen, J. Cala, P. Watson, and A. Romanovsky, "Cost effective, reliable and secure workflow deployment over federated clouds," *IEEE Trans. Serv. Comput.*, vol. 10, no. 6, pp. 929–941, Nov. 2017.

[19] Q. Peng, H. Jiang, M. Chen, J. Liang, and Y. Xia, "Reliability-aware and deadline-constrained workflow scheduling in mobile edge computing," in *Proc. IEEE 16th Int. Conf. Netw., Sens. Control (ICNSC)*, May 2019, pp. 1–6.

[20] L. Han, L.-C. Canon, H. Casanova, Y. Robert, and F. Vivien, "Checkpointing workflows for fail-stop errors," *IEEE Trans. Comput.*, vol. 67, no. 8, pp. 1105–1120, Aug. 2018.

[21] S. Di, Y. Robert, F. Vivien, and F. Cappello, "Toward an optimal online checkpoint solution under a two-level HPC checkpoint model," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 1, pp. 244–259, Jan. 2017.

[22] A. Zhou *et al.*, "Cloud service reliability enhancement via virtual machine placement optimization," *IEEE Trans. Serv. Comput.*, vol. 10, no. 6, pp. 902–913, Nov./Dec. 2017.

[23] H. Han, W. Bao, X. Zhu, X. Feng, and W. Zhou, "Fault-tolerant scheduling for hybrid real-time tasks based on CPB model in cloud," *IEEE Access*, vol. 6, pp. 18616–18629, 2018.

[24] G. Xie, Y. Chen, X. Xiao, C. Xu, R. Li, and K. Li, "Energy-efficient fault-tolerant scheduling of reliable parallel applications on heterogeneous distributed embedded systems," *IEEE Trans. Sustain. Comput.*, vol. 3, no. 3, pp. 167–181, Jul. 2018.

[25] Z. Tong, H. Chen, X. Deng, K. Li, and K. Li, "A scheduling scheme in the cloud computing environment using deep Q-learning," *Inf. Sci.*, vol. 512, pp. 1170–1191, Feb. 2020.

[26] H. Yuan, J. Bi, W. Tan, and B. H. Li, "CAWSAC: Cost-aware workload scheduling and admission control for distributed cloud data centers," *IEEE Trans. Autom. Sci. Eng.*, vol. 13, no. 2, pp. 976–985, Apr. 2016.

[27] Y. Wang and X. Zou, "An effective cloud workflow scheduling approach combining PSO and idle time slot-aware rules," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 5, pp. 1079–1093, May 2021.

[28] H. Yuan, J. Bi, M. Zhou, and A. C. Ammari, "Time-aware multi-application task scheduling with guaranteed delay constraints in green data center," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 3, pp. 1138–1151, Jul. 2018.

[29] R. Pathan, P. Voudouris, and P. Stenström, "Scheduling parallel real-time recurrent tasks on multicore platforms," *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 4, pp. 915–928, Apr. 2018.

[30] K. M. Tarplee, R. Friese, A. A. Maciejewski, H. J. Siegel, and E. K. P. Chong, "Energy and makespan tradeoffs in heterogeneous computing systems using efficient linear programming techniques," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 6, pp. 1633–1646, Jun. 2016.

[31] X. Lin, Y. Wang, Q. Xie, and M. Pedram, "Task scheduling with dynamic voltage and frequency scaling for energy minization in the mobile cloud computing environment," *IEEE Trans. Serv. Comput.*, vol. 8, no. 2, pp. 175–186, Mar./Apr. 2015.

[32] Z. Tang, L. Qi, Z. Cheng, K. Li, S. U. Khan, and K. Li, "An energy-efficient task scheduling algorithm in DVFS-enabled cloud environment," *J. Grid Comput.*, vol. 14, no. 1, pp. 55–74, Mar. 2016.

[33] M. Safari and R. Khorsand, "Energy-aware scheduling algorithm for time constrained workflow tasks in DVFS-enabled cloud enviroment," *Simul. Model. Pract. Theory*, vol. 87, pp. 311–326, Sep. 2018.

[34] L. Shi, Z. Zhang, and T. Robertazzi, "Energy-aware scheduling of embarrassingly parallel jobs and resource allocation in cloud," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 6, pp. 1607–1620, Jun. 2017.

[35] Q. Zhao, C. Xiong, C. Yu, C. Zhang, and X. Zhao, "A new energy-aware task scheduling method for data-intensive applications in the cloud," *J. Netw. Comput. Appl.*, vol. 59, pp. 14–27, Jan. 2016.

[36] L. Wang *et al.*, "Energy-aware parallel task scheduling in a cluster," *Future Generat. Comput. Syst.*, vol. 29, no. 7, pp. 1661–1670, Sep. 2013.

[37] K. Li, "Energy and time constrained task scheduling on multiprocessor computers with discrete speed levels," *J. Parallel Distrib. Comput.*, vol. 95, pp. 15–28, Sep. 2016.

[38] Y. Wen, Z. Wang, Y. Zhang, J. Liu, B. Cao, and J. Chen, "Energy and cost aware scheduling with batch processing for instance-intensive IoT workflows in clouds," *Future Gener. Comput. Syst.*, vol. 101, pp. 39–51, Dec. 2019.

[39] V. Singh, I. Gupta, and P. K. Jana, "An energy efficient algorithm for workflow scheduling in IaaS cloud," *J. Grid Comput.*, vol. 18, no. 3, pp. 357–376, Sep. 2020.

[40] H. Yuan, J. Bi, and M. Zhou, "Profit-sensitive spatial scheduling of multi-application tasks in distributed green clouds," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 3, pp. 1097–1106, Jul. 2020.

[41] S. Kianpisheh, N. M. Charkari, and M. Kargahi, "Reliability-driven scheduling of time/cost-constrained grid workflows," *Future Gener. Comput. Syst.*, vol. 55, pp. 1–16, Feb. 2016.

[42] L. Zhao, Y. Ren, and K. Sakurai, "Reliable workflow scheduling with less resource redundancy," *Parallel Comput.*, vol. 39, no. 10, pp. 567–585, Oct. 2013.

[43] X. Zhu, J. Wang, H. Guo, D. Zhu, L. T. Yang, and L. Liu, "Faulttolerant scheduling for real-time scientific workflows with elastic resource provisioning in virtualized vlouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 27, no. 12, pp. 3501–3517, Dec. 2016.

[44] C. Thirumalaiselvan and V. Venkatachalam, "A strategic performance of virtual task scheduling in multi cloud environment," *Cluster Comput.*, vol. 22, no. 4, pp. 9589–9597, Jul. 2019.

[45] H. A. Hassan, S. A. Salem, and E. M. Saad, "A smart energy and reliability aware scheduling algorithm for workflow execution in DVFS-enabled cloud environment," *Future Gener. Comput. Syst.*, vol. 112, pp. 431–448, Nov. 2020.

[46] W. Tan and M. Zhou, *Business and Scientific Workflows: A Web Service-Oriented Approach*. Hoboken, NJ, USA: Wiley, 2013.

[47] Q.-H. Zhu, H. Tang, J.-J. Huang, and Y. Hou, "Task scheduling for multi-cloud computing subject to security and reliability constraints," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 4, pp. 848–865, Apr. 2021.

[48] W. Li, Y. Xia, M. Zhou, X. Sun, and Q. Zhu, "Fluctuation-aware and predictive workflow scheduling in cost-effective infrastructure-as-a-service clouds," *IEEE Access*, vol. 6, pp. 61488–61502, 2018, doi: 10.1109/ACCESS.2018.2869827.

[49] Q. Wu, M. Zhou, Q. Zhu, Y. Xia, and J. Wen, "MOELS: Multiobjective evolutionary list scheduling for cloud workflows," *IEEE Trans. Autom. Sci. Eng.*, vol. 17, no. 1, pp. 166–176, Jan. 2019.

[50] P. Zhang and M. Zhou, "Dynamic cloud task scheduling based on a two-stage strategy, *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 772–783, Apr. 2018.

[51] H. Yuan, J. Bi, and M. Zhou, "Spatial task scheduling for cost minimization in distributed green cloud data centers," *IEEE Trans. Autom. Sci. Eng.*, vol. 16, no. 2, pp. 729–740, Apr. 2019.

[52] Q. Wu, F. Ishikawa, Q. Zhu, Y. Xia, and J. Wen, "Deadline-constrained cost optimization approaches for workflow scheduling in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 12, pp. 3401–3412, Aug. 2017.

[53] G. Xie *et al.*, "Minimizing redundancy to satisfy reliability requirement for a parallel application on heterogeneous service-oriented systems," *IEEE Trans. Serv. Comput.*, vol. 13, no. 5, pp. 871–886, Sep. 2020.

[54] G. Xie, Y. Chen, Y. Liu, Y. Wei, R. Li, and K. Li, "Resource consumption cost minimization of reliable parallel applications on heterogeneous embedded systems," *IEEE Trans. Ind. Informat.*, vol. 13, no. 4, pp. 1629–1640, Aug. 2017.

[55] R. P. Dick, D. L. Rhodes, and W. Wolf, "TGFF: Task graphs for free," in *Proc. 6th Int. Workshop Hardw./Softw. Codesign (CODES/CASHE)*, Mar. 1998, pp. 97–101.

[56] B. Hu, Z. Cao, and M. Zhou, "Energy-minimized scheduling of real-time parallel workflows on heterogeneous distributed computing systems," *IEEE Trans. Serv. Comput.*, early access, Jan. 26, 2021, doi: 10.1109/TSC.2021.3054754.

[57] X. Tang, "Reliability-aware cost-efficient scientific workflows scheduling strategy on multi-cloud systems," *IEEE Trans. Cloud Comput.*, early access, Feb. 5, 2021, doi: 10.1109/TCC.2021.3057422.

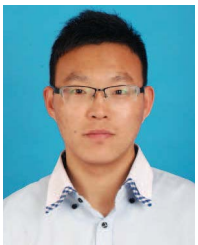
Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.

[58] J. Tang, G. Liu, and Q. Pan, "A review on representative swarm intelligence algorithms for solving optimization problems: Applications and trends," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 10, pp. 1627–1643, Oct. 2021.
[59] Y. Wang, S. Gao, M. Zhou, and Y. Yu, "A multi-layered gravitational search algorithm for function optimization and real-world problems," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 1, pp. 94–109, Jan. 2021.
[60] A. C. Ammari, W. Labidi, F. Mnif, H. Yuan, M. Zhou, and M. Sarrab, "Firefly algorithm and learning-based geographical task scheduling for operational cost minimization in distributed green data centers," *Neurocomputing*, vol. 490, pp. 146–162, Jun. 2022.

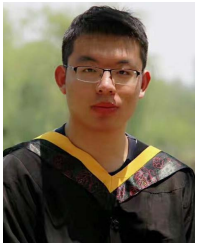
Lingjuan Ye received the B.S. degree from Nanchang University, Nanchang, China, in 2016. She is currently pursuing the Ph.D. degree with the School of Mathematics and Statistics, Beijing Institute of Technology, Beijing, China. Her current research interests cover cloud computing, workflow scheduling, and reinforcement learning.

Yuanqing Xia (Senior Member, IEEE) received the M.S. degree in fundamental mathematics from Anhui University, Hefei, China, in 1998, and the Ph.D. degree in control theory and control engineering from the Beijing University of Aeronautics and Astronautics, Beijing, China, in 2001. He was a Post-Doctoral Research Associate with the Institute of Systems Science, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, from 2002 to 2003. From 2004 to 2006, he was with the University of Glamorgan, Pontypridd, U.K., as a Research Fellow. From 2007 to 2008, he was a Guest Professor with Innsbruck Medical University, Innsbruck, Austria. He is currently a Professor with the School of Automation, Beijing Institute of Technology, Beijing. His research interests are cloud control systems, networked control systems, robust control and signal processing, active disturbance rejection control, unmanned system control, and flight control.

Siyuan Tao received the B.S. degree from the Beijing Institute of Technology, Beijing, China, in 2019, where he is currently pursuing the master's degree with the School of Mathematics and Statistics. His research interests cover cloud computing and workflow scheduling.



Ce Yan is currently pursuing the Ph.D. degree with the Beijing Institute of Technology. His research interests cover cloud control, cloud workflow optimization scheduling, intelligent transportation, actuator saturation, delta operator, and finite frequency.



Runze Gao is currently pursuing the Ph.D. degree with the Beijing Institute of Technology. His research interests cover cloud control systems, cloud resource scheduling, and smart power plant.

Yufeng Zhan received the Ph.D. degree from the Beijing Institute of Technology (BIT), Beijing, China, in 2018. He is currently an Assistant Professor with the School of Automation, BIT. Prior to joining BIT, he was a Post-Doctoral Fellow with the Department of Computing, The Hong Kong Polytechnic University. His research interests include networking systems, game theory, and machine learning.

Authorized licensed use limited to: BEIJING UNIVERSITY OF POST AND TELECOM. Downloaded on March 23,2026 at 12:56:28 UTC from IEEE Xplore. Restrictions apply.